Развёртывание веб-стека на Linux-сервере (Production Deployment)

SSH, firewall, Nginx Reverse Proxy, HTTPS, healthcheck, обновления и резервные копии

Цель и результаты занятия

- Подготовить Linux-сервер для размещения Compose-приложения с минимально необходимым сетевым доступом.
- Опубликовать API через Nginx и объяснить работу reverse proxy и HTTPS.
- Применять healthcheck и restart policy; безопасно обновлять сервис и выполнять откат.
- Создавать резервную копию PostgreSQL и проверять восстановление в отдельной БД.

Учебные вопросы

№	Название
1	Подготовка удалённого Linux-сервера
2	Nginx Reverse Proxy и HTTPS
3	Эксплуатация: обновления, healthchecks, backup и restore

Исходный проект: что уже сделано

- Из лабораторных №7–11: nginx → api (Express) → db (PostgreSQL 17).
- Сервисы находятся в пользовательской сети Docker; PostgreSQL использует именованный том.
- API предоставляет GET /api/products; Nginx перенаправляет запросы к api:3000.
- В теме №12 размещаем уже написанное приложение на отдельном Linux-хосте.

Development и учебный deployment

Локальная разработка

- Проверка на одном компьютере.
- Можно быстро менять и пересобирать образы.
- Открытые порты ради удобства отладки.

Серверный запуск

- Явный входной порт и firewall.
- Секреты не помещаются в образ и Git.
- Резервные копии и проверка восстановления.
- Контроль обновлений, доступа и логов.

Подготовка удалённого Linux-сервера

SSH, firewall, Docker Engine, Compose V2 и права доступа

Серверная среда и границы ответственности

- Пример: Debian 13 или поддерживаемый Ubuntu Server, отдельная виртуальная машина / VPS.
- Docker Engine управляет контейнерами, Compose V2 описывает проект.
- Публичный IP и домен нужны для выхода из Интернета и публичного HTTPS; для лабораторной хватит LAN и HTTP.
- SSH доступен лишь администраторам; секреты приложения храним отдельно от исходного кода.

Схема: Linux-сервер и границы доступа к Docker-стеку

Как организован доступ к приложению и контейнерам на сервере Debian/Ubuntu

Внешние пользователи и администратор



Пользователь

Использует браузер для доступа к веб-приложению

HTTP/HTTPS (порт 80 / 443)



Администратор

Управляет сервером через SSH

SSH :22 (только для администратора)

Linux-сервер

Debian / Ubuntu

Debian Ubuntu

Firewall / UFW

Разрешены только нужные порты (например, 22, 80/8080, 443)

Docker Engine / Docker Compose

Запуск и управление контейнерами

Docker network (внутренняя сеть)

Сервисы доступны только внутри Docker-сети

Nginx
Reverse Proxy
Публичный доступ

Express API
Внутренний сервис

PostgreSQL
База данных

Docker Volume: pgdata
Постоянное хранение данных БД

Границы доступа

Публичный доступ
Только Nginx (Reverse Proxy)
Порт 80 (или 8080) / HTTPS 443
→ Доступ из интернета для пользователей

Административный доступ
SSH (порт 22)
docker compose, logs, backup
→ Только для администратора с SSH-ключами

Внутренний доступ
Express API и PostgreSQL доступны только внутри Docker network
→ Нет прямого доступа из интернета

Что разрешено / Что запрещено

Правильно

- Открывать наружу только Nginx (Reverse Proxy)
- Использовать SSH-ключи для доступа
- Ограничивать firewall (UFW)
- Хранить данные в Docker Volume
- Держать БД без публикации порта наружу

Неправильно

- Публиковать PostgreSQL в интернет без необходимости
- Давать всем доступ к Docker socket
- Использовать root по SSH
- Хранить БД без резервных копий
- Открывать лишние порты

Примеры портов и доступов

Порт / протокол	Сервис	Кому доступен	Назначение
22 / tcp	SSH	Только администратор	Управление сервером (SSH, docker compose, логи, бэкап)
80 / tcp или 8080 / tcp	Nginx (Reverse Proxy)	Пользователи (из интернета)	Доступ к веб-приложению (HTTP)
443 / tcp	Nginx (Reverse Proxy)	Пользователи (из интернета)	Доступ к веб-приложению (HTTPS)
3000 / tcp	Express API	Только внутри Docker network	Внутренний сервис (не публикуется наружу)
5432 / tcp	PostgreSQL	Только внутри Docker network	База данных (не публикуется наружу)

План первоначальной подготовки

1. **Шаг 1** — Подготовить чистый сервер и обновления ОС.
2. **Шаг 2** — Создать административную учётную запись; настроить SSH-ключ.
3. **Шаг 3** — Проверить вход по ключу во второй сессии, только затем менять парольный доступ.
4. **Шаг 4** — Настроить firewall с обязательным разрешением используемого порта SSH.
5. **Шаг 5** — Установить Docker Engine и плагин Compose V2 по официальной инструкции.

Проверка системы и SSH

BASH

```
cat /etc/os-release
ip -br a
ss -tln
systemctl status ssh
```

Что важно

- Установите точную ОС и IP-адрес, убедитесь, что служба SSH активна.
- Проверьте используемый SSH-порт; по умолчанию 22/tcp, но он может быть изменён.

Создание SSH-ключа на рабочем ПК

BASH

```
ssh-keygen -t ed25519 -C "docker-course"
ssh-copy-id student@SERVER_IP
ssh student@SERVER_IP
```

Что важно

- SERVER_IP заменяется адресом учебной VM; команда ssh-copy-id доступна на Linux-клиентах.
- Не отключайте парольную аутентификацию до проверки успешного входа по ключу.

Проверка firewall на сервере

BASH

```
sudo ufw status verbose
sudo ufw allow 22/tcp
sudo ufw allow 8080/tcp
sudo ufw status numbered
```

Что важно

- Команды действуют, если ufw установлен; на Debian пакет может отсутствовать.
- Если SSH использует нестандартный порт — разрешите именно его ДО включения firewall.
- Для публичного HTTPS понадобятся 80/tcp и 443/tcp; 8080 — учебный HTTP-порт.

Ограничение доступа к Docker

- Команды docker требуют доступа к сокету Docker Daemon.
- Членство в группе docker даёт фактически root-права на хосте.
- В учебном проекте используем sudo docker compose ..., без добавления пользователей в группу docker.
- Не публикуем TCP API Docker Daemon без отдельной защищённой конфигурации.

Проверка Docker и Compose

BASH

```
sudo systemctl is-active docker
sudo docker version
sudo docker compose version
sudo docker info --format "{{.ServerVersion}}"
```

Что важно

- Успешная проверка: служба active, клиент видит сервер Docker, версия Compose отображается.
- Если Docker отсутствует, установить из официального репозитория, как в лабораторной №2.

Правила размещения проекта

- Разместить проект, например, в /opt/compose-course12 с контролируемыми владельцем и правами.
- Не копировать в серверный каталог .git, локальные node_modules и файлы с чужими секретами.
- Хранить .env отдельно от публичного frontend и Dockerfile; не публиковать его через Nginx.
- Перед запуском выполнить docker compose config -q; не печатать полный config в отчёт с секретами.

Nginx Reverse Proxy и публикация приложения

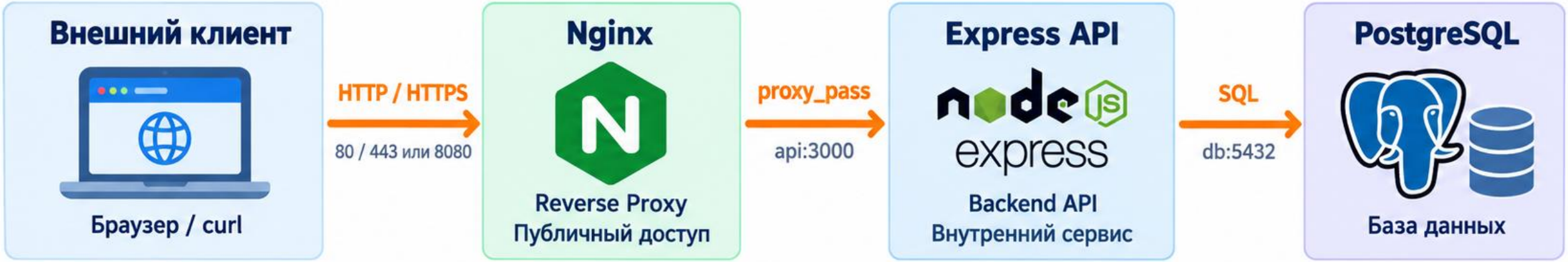
Порты, proxy_pass, заголовки и TLS

Почему один публичный вход через Nginx

- Публикуем только Nginx на хосте; сервисы api и db остаются во внутренней сети Compose.
- Nginx принимает HTTP-запрос и перенаправляет на имя сервиса api:3000.
- При необходимости выполняет TLS termination и раздаёт статические файлы.
- Reverse proxy сам по себе не заменяет защиту приложения, аутентификацию и обновления.

Внешний клиент → Nginx → Express API → PostgreSQL

Путь пользовательского HTTP-запроса через reverse proxy, backend и базу данных



Что делает каждый уровень

Внешний клиент
Отправляет HTTP-запрос

Nginx
Принимает внешний трафик, проксирует запросы

Express API
Обрабатывает маршруты, бизнес-логику и JSON

PostgreSQL
Хранит и возвращает данные

Пример запроса



Что важно понять

- Снаружи публикуется только Nginx
- API и PostgreSQL работают во внутренней сети Docker
- Express API подключается к БД по имени сервиса
- PostgreSQL не нужно открывать в интернет
- Ответ пользователю возвращается по той же цепочке обратно

Правильно

- Открывать наружу только Nginx
- Использовать внутренние DNS-имена сервисов
- Держать БД без публикаций порта наружу

Неправильно

- Подключать клиент напрямую к PostgreSQL
- Использовать localhost вместо имени контейнера между сервисами
- Публиковать все внутрение порты наружу

Nginx: простой reverse proxy

NGINX

```
server {
    listen 80;
    server_name _;
    location /api/ {
        proxy_pass http://api:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Что важно

- Если proxy_pass указан без завершающего /, URI /api/products сохраняется при пересылке.
- Сервис api доступен по DNS внутри Compose-сети.
- Заголовкам X-Forwarded-* доверяют только при корректной конфигурации доверенных прокси.

Проверка маршрута через Nginx

BASH

```
sudo docker compose exec nginx nginx -t
curl -i http://127.0.0.1:8080/api/products
sudo docker compose logs --tail=30 nginx api
```

Что важно

- nginx -t проверяет конфигурацию; код 200 и JSON указывают на доступность API.
- Ошибка 502 требует проверки контейнера api, его порта, DNS и логов.

Порты в учебном и публичном вариантах

Сервис	На хосте	Внутри Compose
Nginx (LAN)	8080/tcp	80/tcp
Nginx (публичный HTTPS)	80 и 443/tcp	80 и 443/tcp
Express API	не публикуем	3000/tcp
PostgreSQL	не публикуем	5432/tcp

Домен и условия выдачи TLS-сертификата

- Для публичного доверенного сертификата домен должен указывать на сервер.
- Для HTTP-01 проверки обычно нужен доступ по 80/tcp из Интернета.
- В локальной сети без публичного домена используют HTTP для демонстрации или локальный доверенный учебный CA.
- Нельзя обещать успешный выпуск сертификата на адрес localhost или произвольный IP.

HTTP, HTTPS и завершение TLS на Nginx

1 HTTP — без шифрования



2 HTTPS — TLS завершается на Nginx (TLS Termination)



1. Защищённое соединение с клиентом

Трафик между браузером и Nginx шифруется с помощью TLS (HTTPS).

2. Завершение TLS на Nginx

Nginx использует SSL-сертификат (например, от Let's Encrypt) для расшифровки запросов.

3. Внутренняя сеть Docker

Между Nginx, Express API и PostgreSQL используется обычный HTTP во внутренней сети Docker (дополнительное шифрование не требуется).

Публичный HTTPS: схема операций

1. **Шаг 1** — Установить DNS-запись домена на публичный IP сервера.
2. **Шаг 2** — Разрешить входящий TCP 80 и 443 в firewall и панели VPS.
3. **Шаг 3** — Настроить веб-сервер для домена и пройти проверку владения доменом через ACME/Certbot.
4. **Шаг 4** — Смонтировать сертификат и приватный ключ в Nginx только для чтения.
5. **Шаг 5** — Проверить HTTPS, срок действия сертификата и процедуру автоматического обновления.

Что делает Certbot

- ACME-клиент запрашивает сертификат у удостоверяющего центра и проходит проверку домена.
- Certbot на хосте и Nginx в контейнере требуют согласовать способ проверки, файлы сертификата и их обновление.
- Выпуск сертификата не осуществляется одной командой, одинаковой для любого Compose-стека.
- Для учебной работы без домена этап HTTPS демонстрирует преподаватель на подготовленном стенде.

Диагностика HTTPS при наличии домена

BASH

```
curl -I http://example.org
curl -I https://example.org
openssl s_client -connect example.org:443 -servername example.org </dev/null
```

Что важно

- Используйте только домен, которым управляете; example.org — placeholder.
- Проверьте DNS, доступность портов, цепочку сертификатов и срок действия.

Эксплуатация Docker Compose на сервере

Restart policies, healthchecks, обновления, backup и restore

Почему running не равно ready

- running означает, что контейнер и основной процесс запущены.
- healthy означает, что заданная проверка здоровья завершалась успешно.
- Запущенный PostgreSQL может ещё не быть готов принимать соединения.
- depends_on с condition: service_healthy позволяет дождаться здорового состояния зависимости при старте.

Healthcheck PostgreSQL в Compose

```yaml
db:
  image: postgres:17
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U $${POSTGRES_USER} -d $${POSTGRES_DB}"]
    interval: 10s
    timeout: 5s
    retries: 5
    start_period: 20s
```

Что важно

- $${...} предотвращает преждевременную подстановку Compose: переменная раскрывается внутри контейнера.
- pg_isready проверяет готовность сервера принимать подключения, но не гарантирует выполнение всех запросов приложения.

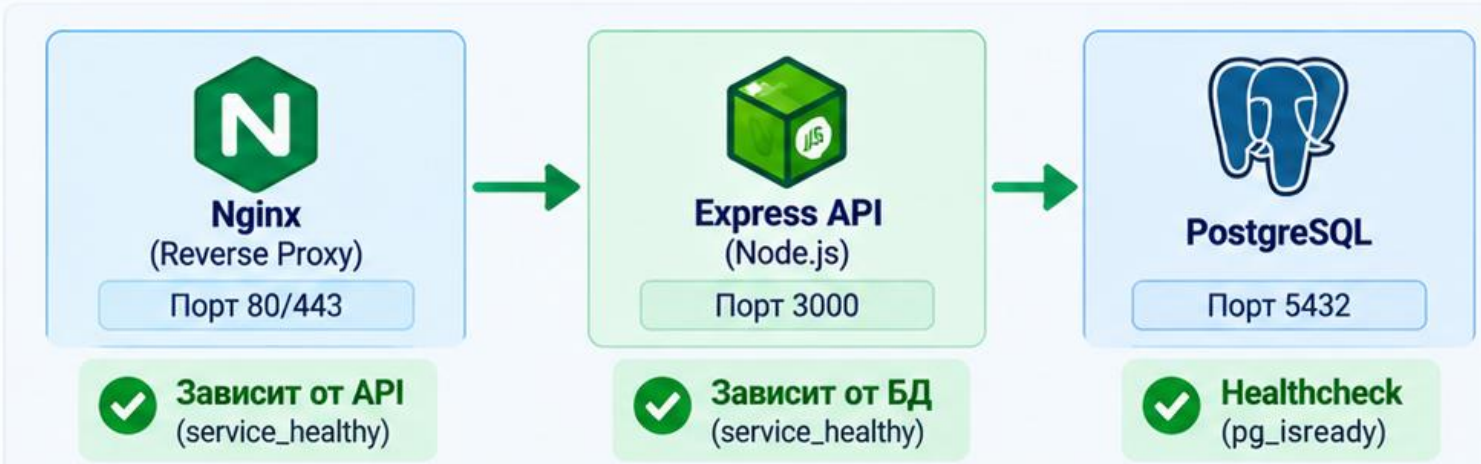
Ожидание готовности БД

```yaml
api:
  depends_on:
    db:
      condition: service_healthy
  restart: unless-stopped
```

Что важно

- Команда up дожидается состояния healthy у db перед созданием api.
- restart: unless-stopped — политика перезапуска контейнера, но не замена healthcheck.

Healthcheck и зависимость сервисов Compose



Пример docker-compose.yml

```yaml
vervion: "3.9"
services:
  db:
    image: postgres:17
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER}"]
      interval: 5s
      timeout: 5s
      retries: 5
  api:
    build: ./api
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 10s
      timeout: 5s
      retries: 5
  nginx:
    image: nginx:latest
    depends_on:
      api:
        condition: service_healthy
    ports:
      - "80:80"
      - "443:443"
```

1. Что такое healthcheck?

- Проверка работоспособности сервиса внутри контейнера.
- Выполняется периодически указанной командой.
- Контейнер получает статус: `starting`, `healthy` или `unhealthy`,
- Compose может использовать этот статус для управления зависимостями сервисов.

2. Зависимость сервисов (depends_on)

- Параметр `depends_on` с условием `service_healthy` заставляет запускать зависимый сервис только после того, как другой сервис стал healthy.
- Полезно, чтобы API не запускался, пока база данных не готова к работе.
- В отличие от простого `depends_on` (без условия), Compose будет ждать, а не просто запускать контейнеры параллельно.

3. Последовательность запуска сервисов



Состояния и диагностика

BASH

```
sudo docker compose ps
sudo docker compose logs --tail=40 api db
sudo docker inspect --format "{{.State.Health.Status}}" CONTAINER_ID
```

Что важно

- Состояние health выводится только у контейнеров с настроенным healthcheck.
- Проверяйте как HTTP-ответ, так и внутренние журналы.

Стратегия управляемого обновления

- Фиксировать версию образа вместо неявного latest.
- Сначала сохранить резервную копию и убедиться, что есть проверенный способ восстановления.
- Для собственных образов выполнять build, для образов из реестра — pull.
- docker compose up -d может пересоздать сервис и вызвать простой: zero downtime не гарантируется.

Обновить только API с пересозданием

```bash
sudo docker compose build api
sudo docker compose up -d --no-deps api
sudo docker compose ps
curl -f http://127.0.0.1:8080/api/products
```

Что важно

- --no-deps позволяет не пересоздавать соседние сервисы.
- Для отката нужен сохранённый предыдущий тег/образ и конфигурация.
- Перед эксплуатационным обновлением проверьте совместимость схемы БД.

Порядок отката изменений

1. **Шаг 1** — Зафиксировать предыдущую рабочую версию образа и конфигурации.
2. **Шаг 2** — Развернуть новую версию API и проверить health/API.
3. **Шаг 3** — При ошибке вернуть предыдущую версию и выполнить docker compose up -d.
4. **Шаг 4** — Сверить логи и HTTP-проверку после возврата.
5. **Шаг 5** — Откат образа не откатывает автоматически миграции схемы или данные PostgreSQL.

Постоянный том не является резервной копией

- Volume переживает пересоздание контейнера, но не защищает от случайного удаления данных и аппаратного сбоя.
- pg_dump создаёт логическую резервную копию данных PostgreSQL.
- Резервная копия должна находиться вне контейнера и вне единственной копии данных.
- Успешный pg_dump нужно дополнить тестом восстановления.

pg_dump: дамп базы на хост

BASH

```
mkdir -p backups
sudo docker compose exec -T db pg_dump -U shopuser -d shop -Fc > backups/shop.dump
ls -lh backups/shop.dump
```

Что важно

- Формат -Fc — архив PostgreSQL для pg_restore.
- Ключ -T нужен для перенаправления вывода в файл на хосте.
- Параметры shopuser/shop берутся из конфигурации проекта; замените их при отличии.

Проверка содержимого дампа

```bash
sudo docker compose exec -T db pg_restore -l < backups/shop.dump | head -30
```

Что важно

- Команда показывает оглавление архива и позволяет убедиться, что файл читается.
- Сама по себе эта проверка не подтверждает успешное восстановление всех данных.

Восстановление в отдельную тестовую БД

```
sudo docker compose exec -T db createdb -U shopuser shop_restore
sudo docker compose exec -T db pg_restore -U shopuser -d shop_restore < backups/shop.dump
sudo docker compose exec -T db psql -U shopuser -d shop_restore -c "SELECT COUNT(*) FROM products;"
```

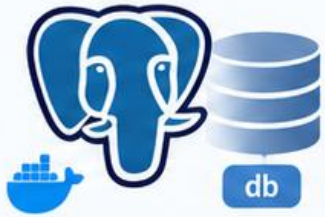
Что важно

- Тестовая БД shop_restore создаётся один раз; при повторении используйте новое имя или согласованный сброс.
- Восстановление в отдельную БД не изменяет рабочую shop.
- Сравните данные рабочей БД и восстановленной.

Резервная копия PostgreSQL и проверка восстановления

Как создать backup базы данных в контейнере и убедиться, что восстановление действительно работает

1 Рабочая база PostgreSQL



Контейнер `db`, база `shop`

2 Создание резервной копии

```
docker compose exec -T db
pg_dump -U shopuser -d shop
> shop.dump
```

Файл backup сохраняется на хосте

3 Проверка наличия backup



`shop.dump`

4 Восстановление в тестовую БД

```
docker compose exec -T db
createdb -U shopuser shop_restore

docker compose exec -T db
psql -U shopuser -d shop_restore
< shop.dump
```

5 Проверка результата



```
SELECT * FROM products;
```

Данные восстановлены

Что резервируем

- Данные таблиц
- Структура БД
- Записи **products**
- Содержимое учебной базы **shop**

Что важно понять

- Backup считается полезным только после проверки восстановления
- Файл dump хранится вне контейнера
- Восстанавливать лучше в отдельную тестовую базу
- Docker Volume не заменяет резервное копирование

Правильно

1. Сначала создать dump, затем проверить восстановление
2. Хранить backup на хосте или внешнем хранилище
3. Проверять данные после restore

Неправильно

1. Считать Docker Volume резервной копией
2. Никогда не проверять восстановление
3. Хранить единственную копию внутри контейнера

Мини-алгоритм

1. pg_dump
2. проверить файл
3. createdb shop_restore
4. psql < shop.dump
5. SELECT для проверки

Команды и назначение

Задача	Команда
Проверка сервисов	docker compose ps
Логи	docker compose logs --tail=50
Проверка конфигурации	docker compose config -q
Обновление собственного API	docker compose build api; docker compose up -d --no-deps api
Логическая копия PostgreSQL	pg_dump -Fc
Проверка восстановления	createdb + pg_restore + SELECT

Что запрещено делать без проверки

- Не применять docker compose down -v к проекту с единственной копией данных.
- Не открывать PostgreSQL 5432 и Express 3000 в Интернет без отдельной необходимости.
- Не копировать секреты и дампы базы в публичные каталоги Nginx.
- Не отключать SSH-доступ до проверки рабочего способа входа.
- Не считать успешное выполнение up доказательством готовности приложения.

Итоги лекции

1. Linux-сервер должен быть защищён на уровнях SSH, firewall, Docker и файлов проекта.
2. Веб-стек публикуется через Nginx; API и БД остаются во внутренней Compose-сети.
3. HTTPS требует реального домена/проверки владения и сопровождения сертификатов.
4. Healthcheck диагностирует готовность, restart policy управляет перезапуском.
5. Обновление Compose может вызвать простой; необходимы проверки и план отката.
6. Резервное копирование PostgreSQL обязательно дополняется тестом восстановления.

Контрольные вопросы

1. Почему контейнерная группа docker считается привилегированной?
2. Чем опубликованный порт Nginx отличается от внутреннего порта PostgreSQL?
3. Почему нельзя получить обычный сертификат Let’s Encrypt для localhost?
4. Что проверяет pg_isready и что он не гарантирует?
5. Чем отличаются restart: unless-stopped и healthcheck?
6. Почему docker compose up -d не гарантирует zero downtime?
7. Зачем использовать pg_dump -Fc и отдельную базу shop_restore?
8. Чем Docker Volume отличается от резервной копии?